NOMAD Agent OS 사용 가이드

nomad-agent-os (https://github.com/DevOps0906/nomad-agent-os)

nomad-blueprints-module (https://github.com/DevOps0906/nomad-blueprints-module)

nomad-secure-pack-module (https://github.com/DevOps0906/nomad-secure-pack-module)

nomad-docker-module (https://github.com/DevOps0906/nomad-docker-module)

목차

1. NOMAD란
2. 레포 구성
3. 설치
4. 프로젝트 적용
5. 팀원 12명
6. 명령어
7. 기본 작업 흐름
8. 모듈 개요
9. 모듈 사용법
10. 모듈 조합 레시피
11. docker 모듈
12. 공통 계약 (contracts)
13. 보안 · 안전장치
14. 점검 · 문제 해결
15. 새 모듈 만들기

작성일 2026-10-28 · 대상: NOMAD를 처음 쓰는 사람 · 이 문서 하나로 설치부터 모듈 조합, 새 모듈 만들기까지 따라 할 수 있습니다.

장	내용
1~2	NOMAD가 무엇이고 레포가 어떻게 나뉘는가
3~4	설치(PC마다 1회) · 프로젝트 적용(프로젝트마다 1회)
5~7	팀원 · 명령어 · 기본 작업 흐름
8~12	모듈(청사진) 개요 · 사용법 · 조합 레시피 · docker · 공통 계약
13~15	보안 · 문제 해결 · 새 모듈 만들기

---

1. NOMAD란

NOMAD Agent OS는 Claude Code를 **AI 개발팀**처럼 쓰게 해 주는 설정 모음입니다. 사용자가 여는 주 세션이 팀장(Opus 5.5)이 되어 일을 나누고, 팀원 12명(Claude 10명 + Codex 중계 2명)이 기획 · 디자인 · 구현 · 테스트 · 독립 검증을 맡습니다. 기획 · 디자인은 Sonnet과 Codex가 초안을 나눠 쓰고, 구현은 Sonnet, 검증은 Codex가 따로 합니다. 같은 파일을 반복해서 읽지 않게 막고, 그림 · 문서 뼈대는 모델 대신 스크립트가 만들어 토큰을 아낍니다. ChatGPT(Codex) · Gemini CLI도 같은 규칙(`AGENTS.md`)을 읽습니다. 도메인 기획 청사진(CRM · LMS 등)은 본체에 넣지 않고 **모듈**로 필요할 때만 가져옵니다.

팀 구조

직급	팀원	모델	하는 일
고문	advisor	Fable 5.1	같은 문제로 2번 실패했을 때만 근본 원인 진단
팀장	(주 세션)	Opus 5.5	나누고 · 맡기고 · 판단. 한 줄 수정은 직접
검증	challenger	Opus 5.5	결제 · 인증 · DB 같은 위험한 계획을 착수 전에 공격
대리	builder · planner · designer · tester · reviewer	Sonnet 5.5	구현 · 문서 · 디자인 · 테스트 · 리뷰
공동 디자인 · 독립 검증	codex-designer · codex-reviewer	Codex (중계는 Haiku 5.5)	UX 초안 · 교차 검토 / 코드 · 디자인 · 접근성 · 보안 독립 검증
사원	scout · intern · diagrammer	Haiku 5.5	찾기 · 반복 작업 · 그림

원칙: 팀원끼리 서로 부르지 않고(깊이 1), commit · push는 팀장만 합니다.

---

2. 레포 구성

설치는 본체 하나, 모듈은 별도 레포에서 필요할 때 가져옵니다.

이름	역할	GitHub	기본 브랜치	공개 범위
nomad-agent-os	본체 — 팀 · 스킬 · 명령 · 도구 · 검사 규칙 · 모듈 등록 목록 (`modules.json`)	https://github.com/DevOps0906/nomad-agent-os	main	비공개
nomad-blueprints-module	도메인 기획 청사진 · 공통 계약(화면 · 상태 · 흐름 · API · 데이터 표)	https://github.com/DevOps0906/nomad-blueprints-module	main	비공개
nomad-secure-pack-module	보안 L3 납품 · 규제 자료 (현재 뼈대)	https://github.com/DevOps0906/nomad-secure-pack-module	main	비공개
nomad-docker-module	컨테이너 실행 템플릿 (실행 파일 포함)	https://github.com/DevOps0906/nomad-docker-module	**master**	비공개

- 비공개 레포라 GitHub에 로그인된 PC(Git Credential Manager · `gh auth login`)에서만 가져올 수 있습니다.
- 모듈 내용은 본체에 복사하지 않습니다(같은 내용이 두 곳에 생기면 어긋남). 본체에는 등록 목록만 둡니다.

---

3. 설치

설치는 **PC마다 한 번**입니다. 설치 후에는 **새 터미널**을 엽니다(도구를 이름만으로 실행하기 위해).

준비물

프로그램	메모
Claude Code	필수
git	Windows는 Git for Windows. PowerShell만 써도 필요
Python 3.8 이상	Windows는 python.org 설치본. 설정의 앱 실행 별칭에서 `python.exe` · `python3.exe` 를 끕니다
Node.js	다이어그램 그림 만들 때만
Codex CLI · Gemini CLI	선택 (ChatGPT · Gemini도 쓸 때만)

설치 명령

환경	명령
Windows PowerShell	`git clone https://github.com/DevOps0906/nomad-agent-os.git ~\workspace\dev\nomad-agent-os` 후 `~\workspace\dev\nomad-agent-os\install.ps1`
Git Bash · macOS · Linux	`git clone https://github.com/DevOps0906/nomad-agent-os.git ~/workspace/dev/nomad-agent-os` 후 `bash ~/workspace/dev/nomad-agent-os/install.sh`

- `~\workspace\dev` 는 예시 위치입니다. 저장소를 다른 곳(예: `D:\WorkSpace\DEV`)에 뒀다면 그 경로로 바꿉니다.
- 스크립트 실행이 막히면 한 번만: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
- 이미 있는 파일은 지우지 않습니다. `~/.claude/CLAUDE.md` 에는 한 줄만 추가하고, `settings.json` 에는 훅만 추가하며, 이름이 같은 에이전트 · 스킬 · 명령은 `~/.claude/.nomad-backup/<시각>/` 에 백업 후 교체합니다.

설치 옵션

옵션	하는 일
`--codex`	ChatGPT Codex에도 공통 원칙 추가 (`~/.codex/AGENTS.md`)
`--gemini`	Gemini CLI에도 공통 원칙 추가 (`~/.gemini/GEMINI.md`)
`--no-path`	도구 폴더를 PATH에 등록하지 않음

예: `.\install.ps1 --codex --gemini`

설치되는 것 (`~/.claude`, 모든 프로젝트가 공유)

항목	수량
팀원 (`agents/`)	12명 (codex-designer · codex-reviewer는 Codex CLI가 있을 때만 동작)
스킬 · 슬래시 명령	스킬 9개 · 명령 10개
도구 (`nomad/bin/`) + `nomad-os` 명령 · 모듈 등록 목록	19개 (`nomad-doctor` 기준)
반복 읽기 차단 훅 (`settings.json`)	있음

확인

방법	기대 결과
Claude Code에서 `/agent-check`	전역 설치 · 프로젝트 적용 · 모듈 · 사양 상태를 한 번에 점검. 안전한 것은 `/agent-check fix`로 자동 구성
`token-audit <프로젝트 경로>`	■ 예산 안 ✅
Claude Code에서 `/agents`	팀원 12명이 보임

---

4. 프로젝트 적용

적용은 **프로젝트마다 한 번**입니다. `nomad-os`는 전역 설치 때 만들어지며 어느 폴더에서나 실행합니다.

상황	명령
새 프로젝트	`nomad-os init <프로젝트 경로> --name "<제품명>"` → `cd <프로젝트>` → `git init -b main` → `claude`
기존 저장소	먼저 `git clone …` → `nomad-os init <저장소 경로>` → `cd <저장소>` → `claude` (순서가 반대면 clone이 실패)

만들어지는 파일과 이유

파일	이유
`AGENTS.md`	**가장 중요.** 스택 · 명령 · 규칙의 단일 정본. 모든 AI 도구가 읽음
`CLAUDE.md`	Claude Code용. `AGENTS.md`를 불러옴
`.claude/`	Claude Code 설정(모델 지정 · `.env` 등 읽기 차단) · 영역 규칙
`.codex/`	ChatGPT Codex 설정 (샌드박스 · 승인)
`.gemini/` · `.geminiignore`	Gemini CLI가 `AGENTS.md`를 읽게 하고 `.env` 등을 제외
`tasks/`	작업 표(`README.md`)와 재진입 스냅샷(`NOW.md`) — `/next`가 따라 움직임

안심해도 되는 점

항목	내용
덮어쓰기	이미 있는 파일은 덮어쓰지 않고 필요한 줄만 추가
git	git 명령을 실행하지 않음 (`git init`은 새 폴더에서 직접)
경로	nomad-agent-os 저장소 안의 경로는 거부. 상위 폴더가 없으면 오타로 보고 멈춤

기존 저장소라면 — AGENTS.md를 실제 코드에 맞추기

템플릿의 스택 · 명령(`pnpm dev` 등)은 예시입니다. `/check` · `/next`가 이 명령을 그대로 실행하므로 Claude에게 이렇게 요청합니다.

```
이 저장소를 분석해서 AGENTS.md의 스택 · 명령 표와 7장 영역 규칙 경로를 실제에 맞게 채워줘
```

5. 팀원 12명

팀장이 알아서 나눠 맡기므로 직접 부를 일은 거의 없습니다. `/agents`로 확인합니다.

이름	모델	맡는 일	맡기면 손해
scout	Haiku 5.5	파일 · 코드 위치 탐색, 구조 요약 (읽기 전용)	판단이 필요한 일 — 지도만 돌려줌
intern	Haiku 5.5	규칙이 한 문장인 반복 작업(추출 · 변환 · 일괄 이름 변경)	로직 변경 · 판단 (하지 않음)
diagrammer	Haiku 5.5	Mermaid 구조도 · 흐름도 · 시퀀스 · ERD + `mmd-check`	구현 · 문서 본문
builder	Sonnet 5.5	구현 · 수정 · 리팩터링 (스택 스킬 먼저 읽음)	한 줄 수정 — 위임 비용이 더 큼
planner	Sonnet 5.5	PRD · 명세 · ADR · 작업 분해 섹션 작성	제품 방향 결정 (팀장 · 사용자 몫)
designer	Sonnet 5.5	디자인 토큰 · DESIGN.md · 미리보기	코드 구현
tester	Sonnet 5.5	테스트 작성 · 실행	운영 코드 수정 (하지 않음)
reviewer	Sonnet 5.5	변경 후 적대적 리뷰 (읽기 전용). mode: code · contract · security · doc · design	직접 고치기
challenger	Opus 5.5	위험한 계획을 착수 전에 공격 (읽기 전용)	일상 작업 — 비용만 늘어남
advisor	Fable 5.1	같은 문제 2회 실패 후 근본 원인 진단	일상 작업 · 착수 전 검토 (부르지 않음)
codex-designer	Codex (Haiku 5.5 중계)	(선택) 공동 기획 · UX 초안 작성과 Sonnet 초안의 변경분 교차 검토. `docs/**` · `design/**` 쓰기. Codex CLI 로그인 + `claude mcp add codex -s user -- cmd /c codex mcp-server` 필요 (macOS·Linux는 `cmd /c` 빼기)	Codex 미설치 시 (호출 실패 → `designer`로 대체)
codex-reviewer	Codex (Haiku 5.5 중계)	(선택) 독립 품질 검증 (읽기 전용). `git diff` + 관련 파일만 보고 code · design · a11y · security 지적. 같은 Codex CLI 로그인 + `claude mcp add codex …` 필요	Codex 미설치 시 (호출 실패 → `reviewer`로 대체)

Codex 모델은 `~/.codex/config.toml`의 기본값을 따릅니다. Codex 팀원이 한도 · 미설치로 실패하면 `상태: Codex 불가`를 보고하고, 팀장이 Claude 팀원으로 대체합니다.

팀장이 직접 하는 일: 한 줄 수정, 위임 비용이 더 큰 일.

위임 규격

팀원에게 주는 지시는 5줄, 600자 이내입니다. 문서 내용을 붙여 넣지 않고 경로#장만 넘깁니다. 팀원 보고는 20줄 이내입니다.

줄	내용
목표	한 문장
쓰기 허용	경로 glob (없으면 none)
근거	문서경로#장 또는 화면 ID
완료 조건	실행할 확인 명령 또는 체크 항목
금지	건드리지 말 것

위험도별 검증

위험	예	착수 전	변경 후
낮음	화면 문구 · 스타일 · 문서	—	builder 자가 검증(`q`), 화면이면 codex-reviewer(design)
보통	기능 · API · 컴포넌트	—	codex-reviewer(code)
높음	결제 · 인증 · RLS · 마이그레이션 · 외부 계약 · 새 의존성 · 노출 면 · 키 권한	challenger	codex-reviewer(code) + reviewer(security/contract) → 필요 시 tester

---

6. 명령어

슬래시 명령 (Claude Code에서 입력)

명령	언제	무엇을
`/agent-check` · `/agent-check fix`	설치 직후 · 이상할 때	전역 설치 · 프로젝트 적용 · 템플릿 미작성 · 규칙 경로 · 팀원 이름 충돌 · 모듈 · 사양 최신 · 보안 · 준비물 · 토큰 점검, `fix`는 안전한 것 자동 구성
`/plan 제품명`	처음부터 기획할 때	린 검증 → PRD → 명세 → 작업 분해 (`docs/`에 문서 뼈대 8개, `tasks/`에 작업 목록)
`/design`	기획 뒤 화면 구현 전	디자인 토큰 → 반응형 폰트 → 미리보기 HTML. 확정 전 화면 구현 시작 안 함
`/diagram 무엇`	그림이 필요할 때	구조도 · 흐름도 · 시퀀스 · ERD (예: `/diagram 결제 흐름`, `/diagram erd`)
`/next`	개발 반복 · 쉬었다 이어갈 때	다음 작업 하나를 계획 → 위임 → 검증 → 커밋까지. 끝나면 `/clear` 후 다시 `/next`
`/check`	수시로	린트 · 타입 · 테스트를 돌리고 실패만 요약
`/secure` · `/secure setup [--ci]`	보안 점검 · GitHub 저장소가 생긴 뒤	SBOM · 취약점 · 라이선스 점검 / Dependabot · SECURITY.md · CI SBOM 생성
`/handoff`	모델 한도가 가까울 때 (Claude가 아직 살아 있을 때)	`tasks/NOW.md` 갱신 → `delta --checkpoint` → `handoff codex --dry-run`으로 이관 파일 경로 안내
`/token-audit`	컨텍스트가 무거워졌을 때	항상 읽히는 설정의 토큰 측정과 줄일 곳 제안
`/oss 지시`	단순 텍스트 작업	로컬 LLM(Ollama 등)에 맡기고 결과 파일만 받기

터미널 도구 (모델 대신 스크립트가 처리 — 토큰 0)

도구	하는 일	예
`nomad-os`	어디서든 NOMAD 설치기 · 모듈 실행	`nomad-os init <경로>` · `nomad-os module add funnel .`
`nomad-module`	`nomad-os module`과 같음	`nomad-module list`
`nomad-doctor`	`/agent-check`의 점검 스크립트	`nomad-doctor .` · `nomad-doctor . --fix`
`plan2spec`	화면 표 → 사양 · 구조도 · 흐름도 · 연결도 · 와이어프레임 · 필수 구조 검사 (바뀐 화면만)	`plan2spec` · `plan2spec --strict`
`spec`	화면 · 흐름 · API · 테이블 하나만 조회	`spec S-03` · `spec changed`
`shot`	375 · 768 · 1280 · 1440 스크린샷	`shot design/wireframes`
`design-preview`	토큰 미리보기 · 대비 검사 · 토큰 우회 탐지	`design-preview tokens.css -o design/preview.html`
`secscan`	SBOM · 취약점 우선순위 · 라이선스 · 보안 설정 파일	`secscan all .` · `secscan vuln --kev` · `secscan setup --ci`
`plan-scaffold`	기획 문서 뼈대 8개	`plan-scaffold docs --name 쇼핑몰`
`q`	긴 출력 명령 실행 → 실패 줄만 표시	`q pnpm test`
`codemap` · `delta`	파일 · 함수 지도 / 바뀐 줄 · 바뀐 파일만	`codemap src` · `delta --changed`
`deps2mmd` · `sql2erd` · `mmd-check`	코드 import → 구조도 / SQL → ERD / Mermaid 검증	`sql2erd supabase/migrations -o docs/diagrams/erd.mmd`
`token-audit` · `oss`	상시 로드 토큰 측정 / 로컬 LLM 위임	`token-audit .` · `oss "3줄 요약" log.txt`
`handoff`	모델 한도 때 이관 (모델 없이 터미널에서 실행). Opus만 제한 → `sonnet`, Sonnet · Claude 제한 → `codex`, 돌아올 때 → `claude`	`handoff codex --dry-run` · `handoff sonnet` · `handoff claude`

전체 목록은 [레퍼런스 (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/REFERENCE.md)](https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/REFERENCE.md)를 봅니다.

---

7. 기본 작업 흐름

새 프로젝트 — SDLC 9단계

먼저 `nomad-os init <경로> --name "<제품명>"` → `cd` → `git init -b main` → `claude` 로 적용합니다. 이후 아래 9단계를 따르며, 02~07은 **기능 하나씩 반복**합니다. 단계가 끝날 때마다 `tasks/NOW.md` 가 갱신됩니다.

단계	하는 일	명령 · 담당
01 검증	린 검증(00_LEAN) 통과 전에는 개발 작업을 만들지 않음	`/plan <제품명>` · 제품 방향 질문만 사용자에게 물음
02 기획	PRD 작성. planner와 codex-designer가 초안을 나눠 쓰고 변경분만 교차 검토 1회 (심각한 충돌이면 1회 더, 이후 팀장이 결정)	`/plan`
03 UX	화면 · 흐름 표, 와이어프레임	`/plan` · `/diagram 회원가입 흐름` (선택) · `plan2spec`
04 UI	브리프(대상 · 주된 일 · 분위기 3단어 · 피할 것)에 답 → 토큰 · 미리보기 → 브라우저로 보고 수정 요청 → 확정	`/design`
05 기술설계	API · 데이터 · ERD · 작업 분해. `기획 문서 기준으로 AGENTS.md 채워줘`	`/plan` · `/diagram erd`
06 구현	작업 하나씩 구현 후 자동 커밋	`/next` → `/clear` → `/next` …
07 QA	codex-reviewer가 독립 검증 (Codex를 못 쓰면 reviewer가 대신)	`/next` 에 포함 · `/check`
08 배포	저장소 올리기, 보안 설정	`gh repo create <계정>/<프로젝트명> --private --source . --push` · `/secure setup --ci`
09 개선	결과를 보고 다음 기능을 02부터 다시	`/plan`

모델 한도에 걸리면 명령어의 `/handoff` · `handoff` 로 이어서 작업합니다(Opus만 제한 → Sonnet이 임시 팀장, 그 외 → Codex). 정책 표는 ARCHITECTURE (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/ARCHITECTURE.md) 3.6장을 봅니다.

내 상황별 첫 명령

상황	입력
코드가 이미 있다	`AGENTS.md를 이 저장소에 맞게 채워줘` → `/next`
처음부터 기획한다	`/plan 제품명`
쉬었다가 이어서 한다	`/next` (`tasks/NOW.md`에 적힌 곳에서 이어감)
기획 문서만 있고 코드는 없다	예시: SceneFlow (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/examples/SCENEFLOW.md)

효율 습관

습관	이유
작업 하나 끝나면 `/clear` 후 `/next`	맥락을 비워 토큰을 아낌. 상태는 `tasks/NOW.md`에 남음
이미 읽은 파일을 다시 읽히지 않기	readguard 훅이 재읽기를 막고 바뀐 줄만 전달
긴 출력 명령은 `q`로 감싸기	실패 줄과 요약만 보임. 전체 로그는 `.nomad/logs/`
구조는 `codemap`, 변경은 `delta`	파일 통독 대신 지도 · 바뀐 줄만
문서 뼈대 · 구조도 · ERD는 도구로	`plan-scaffold` · `deps2mmd` · `sql2erd` — 모델은 내용과 판단만
화면은 `spec <ID>`로 하나만	기획 문서 전체를 다시 읽지 않음
같은 문제 2번 실패 → 고문	advisor가 자동 투입됨

---

8. 모듈 개요

모듈은 **코드가 아니라 기획 청사진**입니다(화면 · 상태 · 흐름 · API · 데이터 표 + 도메인 분석). 가져와서 쓰지 않을 행을 지우고, `plan2spec`으로 시안 · 검사를 만든 뒤, 팀이 내 스택으로 구현합니다. 코드를 그대로 가져오지 않는 이유는 오픈소스 CRM · ERP · LMS가 AGPL · GPL 계열이거나 거대한 단일 앱이라, 구조만 벤치마킹하는 편이 안전하기 때문입니다.

아래 표의 버전 · 규모 · requires · integrates는 `nomad-blueprints-module`의 각 `module.json` 기준입니다. 규모는 화면 · 흐름 · API · 테이블 수입니다. docker 모듈만 `nomad-docker-module` 레포(기본 브랜치 `master`)에 있고, 나머지는 모두 `nomad-blueprints-module`에 있습니다.

core — 공통 계약

이름	버전	내용	규모	requires	integrates	레포
contracts	0.2.0	이벤트 봉투 · AI 작업 · 접근 제어 · 표기 규칙의 정본 (12장)	문서 4종	—	—	blueprints

foundation — 공통 기반

이름	버전	내용	규모	requires	integrates	레포
auth-rbac	0.1.0	가입 · 로그인(이메일 · 매직 링크 · 카카오 · 네이버 · Google) · MFA · 세션 · 조직 전환 · 초대 · 역할 4종 · has_permission	12·5·16·11	contracts	admin-audit, notify	blueprints
admin-audit	0.1.0	운영자 콘솔 · 감사 로그 · 사유 입력 내보내기 · 개인정보 요청 기한 추적 · 월간 접속기록 점검	11·4·10·8	contracts	auth-rbac, notify	blueprints
notify	0.1.0	이메일 · 알림톡 · SMS · 인앱 발송 · 템플릿 검수 · 수신 설정 · 동의 · 차단 · 대체 발송 · 웹훅	9·5·13·11	contracts	auth-rbac, payments, funnel	blueprints
payments	0.1.0	토스페이먼츠 일반결제 · 빌링키 정기결제 · 환불 · 웹훅 · 대사 · 구독 상태	12·5·14·12	contracts	lms, notify, admin-audit	blueprints
files-storage	0.1.0	Supabase Storage · tus 재개 업로드 · 형식 검증 · ClamAV 검사 후 공개 · 버전 · 공유 링크 · 휴지통	9·5·13·8	contracts	stt-voice, rag-chat, ocr-docs	blueprints

domain — 도메인

이름	버전	내용	규모	requires	integrates	레포
funnel	0.1.0	랜딩 · 리드 폼 · 동의 · 점수 · 배정 · 육성 시퀀스 · 파이프라인 · 퍼널 대시보드	12·5·11·16	contracts	crm, notify	blueprints
crm	0.1.0	회사 · 연락처 · 활동 타임라인 · 딜 파이프라인 · 견적(품목 · 할인 · VAT · PDF) · 중복 병합 · 가져오기	14·5·17·16	contracts	funnel, erp	blueprints
erp	0.1.0	기준정보 · 구매(발주→입고→매입) · 재고 원장 · 판매(견적→주문→출고→매출) · 회계 연결	20·7·23·23	contracts	crm, ocr-docs	blueprints
lms	0.1.0	강의 · 단원 · 차시 · 수강(무료 · 유료 · 배정) · 진도율 · 퀴즈 · 과제 · 라이브 출석 · 수료증 · 법정대리인 동의	16·6·17·16	contracts	payments, notify, files-storage	blueprints
booking	0.1.0	자원 · 서비스 예약 · DB 배타 제약(EXCLUDE gist)으로 이중 예약 차단 · 결제 대기 홀드 10분 · 공개 예약 페이지(비로그인 · 토큰 링크 변경 · 취소) · 취소 정책 스냅샷 · 리마인더 · ICS 내보내기 · 이벤트 5종(outbox)	9·5·15·10	contracts	payments, notify, funnel, crm	blueprints
helpdesk	0.1.0	웹 문의 폼 · 이메일 수신(인바운드 웹훅 · DMARC 정렬 기록 · 회신 토큰 스레드 매칭 · 루프 방지) · 티켓 상태 · 영업시간 SLA · 공개 답변/내부 메모 분리 · 첨부 · FAQ · 매크로 · CSAT · 3년 경과 가명 처리	11·5·16·12	contracts, files-storage	crm, notify, rag-chat	blueprints

ai — AI

이름	버전	내용	규모	requires	integrates	레포
stt-voice	0.1.0	faster-whisper 기반 음성 받아쓰기 · 동의 기록 · 5분 조각 병렬 처리 · 전사 편집 · 요약 · 내보내기 · AI 작업 계약(정본)	10·5·13·11	contracts, files-storage	rag-chat	blueprints
rag-chat	0.1.0	pgvector 색인 · 한국어 키워드 하이브리드 검색 · 청크 단위 권한 필터 · 출처 필수 답변 · 프롬프트 주입 방지	9·5·13·11	contracts, files-storage	stt-voice	blueprints
ocr-docs	0.1.0	PaddleOCR 자체 호스팅 · 영수증 / 사업자등록증 / 계약서 필드 추출 · 낮은 신뢰도 필드 사람 확인 · 주민번호 저장 전 마스킹	8·4·11·8	contracts, files-storage	erp, crm	blueprints

infra — 실행 환경

이름	버전	내용	규모	requires	integrates	레포
docker	0.1.0	컨테이너 실행 템플릿(api · worker · whisper.cpp · PaddleOCR · TEI 임베딩 · ClamAV, compose profiles). `--allow-scripts` 필요 (11장)	프로필 5	—	—	docker

security — 보안

이름	버전	내용	규모	requires	integrates	레포
secure-pack	0.0.1	L3 납품 · 규제 — OpenChain 5230/18974 · VEX · SPDX · CRA · 오픈소스 정책. 현재 **뼈대 (구조 계획만)**	미확인	—	—	secure-pack

- 모듈마다 ID 접두사(`FN-S01`, `CRM-S01`, `BKG-…`, `HD-…`)가 있어 여러 모듈을 한 프로젝트에 가져와도 화면 ID가 겹치지 않습니다.
- 이 목록은 2026-10-28 기준입니다. 새 버전은 `nomad-os module list` 로 확인합니다.
- `module.json` 의 license가 대부분 `Proprietary` 입니다. 외부에 재배포하지 마세요.

---

9. 모듈 사용법

기본 명령

PowerShell · Git Bash에서 같은 명령입니다.

명령	하는 일
`nomad-os module list [프로젝트]`	등록된 모듈 · 설치 상태 (분류별로 묶어 표시)
`nomad-os module add <이름> <프로젝트>`	가져오기 (커밋 고정)
`nomad-os module update <이름> <프로젝트>`	새 버전 가져오기
`nomad-os module update all <프로젝트>`	설치된 모듈 전부 업데이트
`nomad-os module remove <이름> <프로젝트>`	고치지 않은 파일만 삭제
`nomad-os module add <이름> <프로젝트> --from <로컬 경로>`	레포 없이 로컬에서 (개발 · 시험용)
`nomad-os module add https://github.com/<계정>/<레포>.git#<하위 경로> <프로젝트>`	등록 안 된 레포에서
`nomad-os module add docker <프로젝트> --allow-scripts`	실행 파일 포함 모듈 (11장)

`<프로젝트>`가 현재 폴더면 `.`을 씁니다. 설치 후 `nomad-module`로도 같은 명령을 실행할 수 있습니다.

가져오는 위치

유형	가져오는 곳
blueprint	`docs/blueprints/<이름>/`
security	`security/<이름>/`
docker	`infra/<이름>/`

requires 자동 설치 · integrates 안내

필드	뜻	동작
`requires`	꼭 함께 있어야 하는 모듈 (예: 거의 모든 모듈 → contracts, AI 모듈 · helpdesk → files-storage)	`module add` 때 먼저 자동 설치. 하나라도 검증 실패면 아무것도 설치하지 않음. 실행 파일을 포함한 모듈(docker)은 자동 설치하지 않고 직접 `--allow-scripts`로 받게 안내
`integrates`	같이 쓰면 연결되는 모듈 (설치 안 해도 단독 동작)	설치 끝에 "함께 쓰면 연결: …(설치됨/미설치)" 안내만

- `module remove`는 다른 설치 모듈이 requires로 쓰고 있으면 경고합니다.
- 내 레포를 더하려면 `~/.claude/nomad/modules.local.json`에 같은 형식으로 적습니다(NOMAD 업데이트에 덮이지 않음).

기록 파일 — `nomad-modules.json` (프로젝트 루트, 커밋 권장)

항목	내용
source · ref · commit	어느 레포 · 브랜치 · **커밋**에서 가져왔는지 — 같은 기록으로 팀원 · CI가 같은 내용을 받음
install_to	설치된 위치
files	파일별 sha256 해시
requires	의존 기록 (remove 경고에 사용)

고친 파일 보존 · `.upstream`

상황	동작
내가 안 고친 파일	update 때 새 버전으로 교체
내가 고친 파일 (해시가 다름)	덮어쓰지 않음. 새 버전은 `<파일>.upstream`으로 옆에 둠 — 비교해서 직접 합침
remove	고치지 않은 파일만 삭제

청사진 → 와이어프레임 · 구조도 (`plan2spec`)

```
nomad-os module add funnel .    # → docs/blueprints/funnel/
plan2spec                       # 구조도 · 흐름도 · 연결도 · 와이어프레임 · 필수 구조 검사
plan2spec --strict              # 필수 구조(🔴)가 하나라도 빠지면 실패
spec FN-S01                     # 화면 하나만 조회 (예시 ID)
```

순서	할 일
1	`module add` 로 청사진을 가져온다
2	우리 서비스에 쓰지 않을 행을 지운다
3	`plan2spec` 으로 시안 · 검사. 🔴 이 나오면 도메인 필수 구조가 빠진 것
4	팀이 `/next` 로 내 스택에 맞게 구현

---

10. 모듈 조합 레시피

모든 레시피에서 `contracts`(와 필요한 `files-storage`)는 requires로 **자동 설치**되므로 따로 쓰지 않아도 됩니다. 명령은 프로젝트 폴더에서 `.`로 실행합니다.

레시피	모듈	연결되는 것 (integrates)
SaaS 기본	auth-rbac + admin-audit + notify + payments	가입 → 운영 감사 → 알림 → 결제
온라인 강의	lms + payments + notify + files-storage	수강 결제 · 수료 알림 · 강의 자료
영업	funnel + crm + notify	리드 → 딜 전환 → 후속 알림
사내 문서 AI	rag-chat + files-storage + stt-voice + ocr-docs	업로드 → 검사 → 색인 · 받아쓰기 · 문서 인식
예약 서비스	booking + payments + notify	예약 홀드 → 결제 → 리마인더
고객지원	helpdesk + crm + notify	티켓 → 고객 이력 → 회신 알림

SaaS 기본

```
nomad-os module add auth-rbac .
nomad-os module add admin-audit .
nomad-os module add notify .
nomad-os module add payments .
plan2spec --strict
```

온라인 강의

```
nomad-os module add lms .
nomad-os module add payments .      # 결제는 payments 이벤트 계약으로 연결
nomad-os module add notify .
nomad-os module add files-storage .
plan2spec --strict
```

영업 (퍼널 → CRM)

```
nomad-os module add funnel .
nomad-os module add crm .           # funnel 리드 전환 연결
nomad-os module add notify .
plan2spec --strict
```

사내 문서 AI

```
nomad-os module add files-storage .
nomad-os module add rag-chat .
nomad-os module add stt-voice .
nomad-os module add ocr-docs .
plan2spec --strict
```

실제로 돌리려면 자체 호스팅 서비스가 필요하므로 11장 docker 모듈을 더합니다(`nomad-os module add docker . --allow-scripts`).

예약 서비스

```
nomad-os module add booking .
nomad-os module add payments .
nomad-os module add notify .
plan2spec --strict
```

`booking`은 본체 `modules.json`에 등록된 뒤에 `module add`가 동작합니다(8장 참고).

고객지원

```
nomad-os module add helpdesk .      # files-storage가 requires로 함께 설치됨
nomad-os module add crm .
nomad-os module add notify .
plan2spec --strict
```

`helpdesk`도 등록 후에 동작합니다. 조합이 정해지면 `/plan`에서 "docs/blueprints의 모듈 기준으로 PRD를 채워 줘"처럼 요청해 기획에 반영합니다.

---

11. docker 모듈

blueprints가 정한 자체 호스팅 서비스를 **로컬 · 단일 서버**에서 실행하는 템플릿입니다(쿠버네티스 등 운영 오케스트레이션은 범위 밖). 이 compose는 외부 API를 호출하지 않습니다(STT · OCR · 임베딩 모두 자체 호스팅).

설치

```
nomad-os module add docker <프로젝트> --allow-scripts
```

→ 프로젝트 `infra/docker/`. **설치 전 `compose.yml` · `Dockerfile`을 직접 검토합니다.** 셸 스크립트(`.sh` 등)는 플래그가 있어도 거부되고, Dockerfile · .dockerignore · .env.example · .conf · .toml만 추가 허용됩니다.

프로필

프로필	서비스	용도
core	api · worker	FastAPI 앱 + 잡 워커
security	clamd · clamav-update	files-storage 악성코드 검사
ai-cpu	stt-whisper-cpu · ocr-paddle-cpu · embeddings-cpu	GPU 없이 STT · OCR · 임베딩
ai-gpu	stt-whisper-gpu · embeddings-gpu	NVIDIA GPU (OCR GPU는 0.1.0 제외)
fetch	models-fetch · models-fetch-ocr	모델 받기(일회성)

`ai-cpu`와 `ai-gpu`는 함께 켜지 않습니다(네트워크 별칭 `stt` · `embed` 중복).

시작 순서

순서	할 일
1	`supabase start` (Supabase는 compose에 없음). 출력의 URL · anon · service_role 키를 `.env` (`.env.example` 복사)에 넣음
2	앱 코드 준비: `apps/api` (api · worker 공용, `requirements.txt` 필요), OCR 사용 시 `apps/ocr-worker`
3	모델 받기(최초 1회, AI 사용 시): `.env` 에 `BGE_M3_REVISION` (필수) 지정 후 `docker compose --profile fetch run --rm models-fetch` (OCR은 `models-fetch-ocr`)
4	STT 사용 시 `.env` 에 `STT_IMAGE_CPU` · `STT_IMAGE_GPU` 지정(기본값 없음, 고정 태그 또는 digest)
5	실행: `docker compose --profile core --profile security up -d` (AI는 `--profile ai-cpu` 또는 `--profile ai-gpu` 추가)
6	문법 확인(.env 없이): `docker compose -f compose.yml --profile core --profile security --profile ai-cpu --profile ai-gpu --profile fetch config --no-interpolate -q`

모듈 ↔ 서비스

blueprints 모듈	서비스	프로필	접속(내부)
(공통)	api · worker	core	—
files-storage	clamd	security	`clamd:3310`
stt-voice	whisper.cpp 서버	ai-cpu / ai-gpu	`http://stt:8080`
ocr-docs	ocr-paddle	ai-cpu	`http://ocr:8080`
rag-chat	TEI (bge-m3)	ai-cpu / ai-gpu	`http://embed:80`

주의

항목	내용
서비스 키	`SUPABASE_SERVICE_ROLE_KEY`는 **worker에만**. api는 anon 키 + 사용자 JWT
worker 내부 HTTP(8001)	`WORKER_INTERNAL_TOKEN` 필수 (api · worker만 보유, AI 컨테이너에는 없음)
외부 포트	api 하나(`127.0.0.1:8000`)
네트워크	api는 AI · clamd에 직접 닿지 않음. AI · clamd는 내부 네트워크(`nomad-ai`)에만 붙고 오프라인
빌드 전	컨텍스트에 `.env` · 키 파일이 없는지 사람이 최종 확인
이미지	digest 고정 권장 (`docker buildx imagetools inspect <image:tag>`)
GPU	NVIDIA 드라이버 + nvidia-container-toolkit 필요. whisper.cpp `main-cuda`는 amd64만
실측 필요	메모리 제한(clamd 3GB · TEI 4GB · stt 4GB · ocr 3GB · api/worker 512MB)은 제안값. arm64 지원 일부는 `확인 필요`
자동 설치 안 됨	requires로 자동 설치되지 않음 — 직접 `--allow-scripts`로 받음

자세한 문제 해결(clamd 첫 기동 지연, TEI가 bge-m3를 못 읽음, `host.docker.internal` 연결 실패)은 docker 레포 README의 "문제 해결"을 봅니다.

---

12. 공통 계약 (contracts)

여러 모듈에 복사돼 있던 공통 규칙의 **정본 1곳**입니다. 각 모듈은 규칙을 복사하지 않고 `../contracts/<문서>`로 링크합니다. 규칙을 한 곳에서 고치면 모든 모듈이 같이 따라가고, 모듈끼리 이벤트 · 권한 · 표기 방식이 어긋나지 않습니다. 그래서 대부분 모듈의 `requires`이며 함께 가져와야 문서 링크가 열립니다.

문서	한 줄	왜 필요한가
EVENTS	이벤트 봉투 12필드 · outbox/inbox · HMAC · 순서 · 모르는 type 처리	모듈 간 연결(결제 완료 → 수강 등록)이 같은 형식이어야 안 깨짐
JOBS	AI 작업 테이블 · 단계 · 상태 전이 · 재시도=새 작업 · 환불 멱등 키	AI 모듈(stt · rag · ocr)이 같은 작업 모델을 쓰고 중복 실행 · 중복 환불을 막음
ACCESS	has_permission · 플랫폼 운영자 · 서비스 키 · 서버 간 인증 · owner_module	권한 판단과 서비스 키 사용을 한 규칙으로 통일
CONVENTIONS	금액 · VAT · 개인정보 마스킹 · "확인 필요" 표기 · 이벤트 이름 · ID 접두사 표	표기가 모듈마다 달라지는 것을 막음

- 모듈 고유 값(type 목록 · kind · step · payload)은 해당 모듈에 두고, 공통 규칙만 contracts에서 가져옵니다.
- 계약이 바뀌면 minor 버전이 오르고, 참조하는 모듈이 영향을 받는지 확인합니다.

---

13. 보안 · 안전장치

보안 레벨 (프로젝트마다 하나)

레벨	언제	어떻게
L0 기본	모든 프로젝트 (자동)	그냥 쓰면 적용 — 비밀 커밋 금지 · lockfile 커밋 · 새 의존성 점검 · 기능별 위협 메모 · 보안 리뷰
L1 저장소	GitHub 저장소가 생기면	`/secure setup`
L2 배포	배포 파이프라인이 생기면	`/secure setup --ci`
L3 납품 · 규제	공공 · 금융 · 대기업 · EU 판매	`nomad-os module add secure-pack .` (현재 뼈대)

PRD `## 8. 보안·공급망`에 기록하며, 비어 있으면 리뷰에서 막힙니다. 자세히는 보안 가이드 (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/SECURITY.md)를 봅니다.

모듈 가져오기 안전장치

위험	막는 방법
스크립트 · 실행 파일	허용 확장자만(.md .json .yml .csv .txt .svg .png .mmd .css .html). 스크립트는 `--allow-scripts`로 검토 후에만
docker 모듈	Dockerfile · .dockerignore · .env.example · .conf · .toml만 추가 허용, `--allow-scripts` 필수, 셸 스크립트는 거부
requires 조작	이름 형식(`a-z0-9-`)만 · 등록부(또는 같은 `--from` 루트)의 모듈만 · 깊이 8 · 순환 거부. `--allow-scripts`는 requires에 전파 안 됨
설치 위치 조작	`install_to`는 docs/ · security/ · infra/ 아래만, `.git` · `.github` · `.vscode` 경로 거부
경로 조작(`../`) · 심볼릭 링크	모듈 폴더 · 프로젝트 밖을 가리키면 거부
몰래 바뀐 내용	커밋 고정 + 파일 해시 기록
큰 파일	모듈당 5MB 제한
라이선스	`module.json`의 license 확인 — 없으면 경고, GPL · AGPL 등은 🔴
NOMAD 저장소 오염	NOMAD 저장소 안에는 설치 거부

비밀 관리

규칙	내용
읽기 차단	Claude가 `.env` · `*.pem` · `*.key` · SSH 키 파일을 읽지 못하게 막음
커밋 금지	비밀 · 로그에 키 금지, `.gitignore`에 `.nomad/` · `.env` 포함
키 위치	docker 모듈에서 `service_role` 키는 worker에만. `.env`는 `.env.example`을 복사해 만들고 커밋하지 않음
이미지 빌드	빌드 컨텍스트에 `.env` · 키 파일이 없는지 확인

"확인 필요" 표기의 의미

법령 · 시장 · 수치를 원문으로 확인하지 못했다는 뜻입니다. 기획 문서와 청사진에서 *(확인 필요)* 또는 **미확인**으로 적고 근거를 요구하며, 확인 전에는 사실로 쓰지 않습니다. 예: 개인정보보호법 조항 번호, 카드번호 마스킹 범위, docker 모듈의 일부 arm64 지원. 납품 · 출시 전에 원문으로 확인해 표기를 지웁니다.

---

14. 점검 · 문제 해결

nomad-doctor 결과 읽기

`/agent-check`(또는 터미널의 `nomad-doctor .`)는 항목별 표를 보여 줍니다.

표시	뜻	할 일
✅	정상	개수만 확인
⚠️	주의	내용 확인 후 필요하면 조치
❌	문제	표의 해결 안내를 따름
자동으로 고칠 수 있는 것	안전하게 자동 구성 가능한 항목	`/agent-check fix` 또는 `nomad-doctor . --fix` (전역 재설치 · init · plan2spec)

전역 재설치

NOMAD 저장소 폴더에서 실행합니다. 여러 번 실행해도 중복 추가되지 않습니다.

PowerShell	Git Bash
`git pull` → `.\install.ps1`	`git pull && bash install.sh`

- 새로 생긴 설정 파일은 프로젝트에서 `nomad-os init`을 한 번 더 실행하면 추가됩니다.
- `AGENTS.md`는 보존되므로 바뀐 장은 `project-template/AGENTS.md`를 보고 직접 옮깁니다.
- 모듈 등록이 바뀐 경우(예: booking · helpdesk 등록)도 재설치가 필요합니다.

자주 묻는 질문

질문	답
`nomad-os`를 찾을 수 없다고 나옵니다	새 터미널을 열고, 그래도 안 되면 `install.ps1`을 다시 실행하거나 `<저장소 위치>\install.ps1 init <경로>`로 실행합니다
`'~' 모듈을 로드할 수 없습니다`	`~\...\install.ps1` 파일이 그 위치에 없다는 뜻입니다. 실제 저장소 위치로 실행하거나 `nomad-os init …`을 씁니다
`상위 폴더가 없습니다` 오류	프로젝트 경로 오타입니다. 표시된 "해석된 경로"를 보고 실제 경로로 다시 실행합니다
`Git Bash(bash.exe)를 찾지 못했습니다`	Git for Windows를 설치하거나 `$env:CLAUDE_CODE_GIT_BASH_PATH`로 위치를 지정합니다
`token-audit`를 찾지 못합니다	설치 후 새 터미널을 열었는지 확인합니다
쉬었다가 다시 시작하려면?	`/next`만 입력합니다. `tasks/NOW.md`에 적힌 곳에서 이어갑니다
같은 문제가 계속 실패하면?	2번 실패하면 고문(advisor)이 자동 투입됩니다
`module add`가 비공개 레포라 실패합니다	GitHub에 로그인된 PC인지 확인합니다(Git Credential Manager · `gh auth login`)

---

15. 새 모듈 만들기

폴더 형식

blueprints 레포(`nomad-blueprints-module/<이름>/`)에 만듭니다. funnel이 예시입니다.

파일	내용
`module.json`	이름 · 유형 · category · 버전 · 라이선스 · install_to · files · id_prefix · requires · integrates · summary
`README.md`	도메인 분석: 단계 · 지표 정의 · 상태 · 법 · 동의 · 보안 · MVP 범위 · 벤치마킹
`screens.md`	`## 화면` / `## 상태` / `## 흐름` 표 (ux-patterns 2절 형식 그대로)
`api.md`	`| ID | 메서드 | 경로 | 입력 | 출력 | 권한 | 오류 |`
`data.md`	`| 테이블 | 목적 | 주요 컬럼 | RLS 기준 | 비고 |`
`patterns.json`	(선택) 이 도메인에서 추가로 반드시 있어야 하는 구조 — `must_extra` · `applies_to_prefix`

module.json 필드

필드	뜻	예
`name`	모듈 이름 (`a-z0-9-`)	`booking`
`type`	`blueprint` · `security` · `docker`	`blueprint`
`title`	표시 이름	`Booking — 예약·일정`
`category`	core · foundation · domain · ai · infra · security	`domain`
`version`	버전	`0.1.0`
`license`	라이선스 (없으면 경고, GPL · AGPL은 🔴)	`Proprietary`
`nomad`	필요한 NOMAD 버전	`>=1`
`id_prefix`	화면 · API ID 접두사 (모듈끼리 겹치지 않게)	`BKG`
`install_to`	가져올 위치 (docs/ · security/ · infra/ 아래만)	`docs/blueprints/booking`
`files`	포함 파일 목록 (허용 확장자만)	`README.md` , `screens.md` …
`summary`	한 줄 요약 + 규모(화면 · 흐름 · API · 테이블)	—
`requires`	꼭 함께 있어야 하는 모듈	`["contracts"]`
`integrates`	같이 쓰면 연결되는 모듈	`["payments", "notify"]`

- 허용 확장자: `.md .json .yml .csv .txt .svg .png .mmd .css .html` — 스크립트는 넣지 않습니다(가져올 때 차단).
- 공통 규칙(이벤트 · 권한 · 표기)은 복사하지 않고 `../contracts/<문서>`로 링크합니다.

등록과 검증

단계	할 일
1	`nomad-blueprints-module/<이름>/`에 파일을 만든다
2	검증 (아래 명령)
3	커밋 → push → 필요하면 `<이름>-v0.1.0` 태그
4	본체 `modules.json`에 등록 (또는 내 PC만이면 `~/.claude/nomad/modules.local.json`) → 본체 커밋 · push
5	각 PC에서 `install.ps1` 재실행

```
mkdir -p /tmp/t/docs && nomad-os module add <이름> /tmp/t --from <nomad-blueprints-module 경로> && cd /tmp/t && plan2spec --strict
```

`--strict`는 필수 구조(🔴)가 하나라도 빠지면 실패합니다. 통과하면 모듈 형식이 맞는 것입니다.

관련 문서: README (https://github.com/DevOps0906/nomad-agent-os/blob/main/README.md) · 설치 가이드 (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/INSTALL.md) · 시작하기 (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/GETTING_STARTED.md) · 모듈 (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/MODULES.md) · 보안 가이드 (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/SECURITY.md) · 레퍼런스 (https://github.com/DevOps0906/nomad-agent-os/blob/main/docs/REFERENCE.md)